

Docker for ASP.NET Core

The Developer's Guide to Containerizing .NET Applications

1. What is Docker & Why Containerize ASP.NET Core?

Docker is a platform that packages an application and all its dependencies into a single, standardized unit called a container. For ASP.NET Core developers, this is a game-changer.

Why is it Important for .NET?

- **Cross-Platform Consistency:** Develop on Windows or macOS, and deploy flawlessly to Linux environments. Docker ensures the underlying OS and .NET runtime are exactly what you specify.
- **No "It works on my machine":** The container includes the specific version of the .NET runtime (e.g., .NET 8), so you never have to worry if the server has the right framework installed.
- **Microservices Ready:** ASP.NET Core's lightweight nature combined with Docker's fast startup times makes it the perfect pairing for microservices architectures.

2. Core ASP.NET Core Docker Concepts

Multi-Stage Builds

The standard practice in .NET containerization. You use a heavy SDK image to compile and publish the application, and a much smaller Runtime image to actually run it. This keeps the final image tiny and secure.

ASPNETCORE_ENVIRONMENT

A crucial environment variable. ASP.NET Core uses this (e.g., set to `Development`, `Staging`, or `Production`) to determine which `appsettings.{Environment}.json` file to load and how to configure middleware (like showing detailed errors).

Kestrel in Containers

ASP.NET Core uses Kestrel as its cross-platform web server. In Docker, Kestrel typically binds to port 80 or 8080. You map this internal port to an external port on your host machine.

3. Writing an ASP.NET Core Dockerfile

Here is the Microsoft-recommended multi-stage `Dockerfile` for an ASP.NET Core application:

```
# STAGE 1: Build
# Use the SDK image for building the app
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

# Copy csproj and restore dependencies (caches this layer)
COPY ["MyApp/MyApp.csproj", "MyApp/"]
RUN dotnet restore "MyApp/MyApp.csproj"

# Copy the rest of the code and build
COPY . .
WORKDIR "/src/MyApp"
RUN dotnet build "MyApp.csproj" -c Release -o /app/build

# Publish the application
RUN dotnet publish "MyApp.csproj" -c Release -o /app/publish /p:UseAppHost=false

# STAGE 2: Run
# Use the lightweight ASP.NET runtime image
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
WORKDIR /app
EXPOSE 8080

# Copy the published output from the build stage
COPY --from=build /app/publish .

# Define the entry point for the container
ENTRYPOINT ["dotnet", "MyApp.dll"]
```

Explanation & Notes:

- **mcr.microsoft.com/dotnet/...** : Microsoft's official container registry. We use the **sdk** for compilation and **aspnet** for runtime.
- **Caching strategy:** We copy the **.csproj** file and run **dotnet restore** *before* copying the rest of the source code. This ensures that if only your code changes (not your NuGet packages), Docker reuses the cached restore layer, saving massive build time.
- **ENTRYPOINT** : Tells the container to start the .NET runtime and execute your compiled DLL.

4. Managing Environments and Configuration

ASP.NET Core reads configuration from `appsettings.json`, environment variables, and more. When using Docker, passing environment variables is the easiest way to override settings.

You can run your container in Production mode by passing the environment variable:

```
docker run -d -p 8080:8080 -e ASPNETCORE_ENVIRONMENT=Production my-aspnet-app
```

This automatically tells ASP.NET Core to load `appsettings.Production.json` and apply production-safe middleware.

5. Docker Compose with ASP.NET Core & SQL Server

To run your ASP.NET Core app alongside a database, use Docker Compose. Here is a `compose.yaml` example:

```
services:
  webapi:
    build:
      context: .
      dockerfile: MyApp/Dockerfile
    ports:
      - "5000:8080"
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      # Inject the connection string pointing to the 'db' service
      - ConnectionStrings__DefaultConnection=Server=db;Database=master;User=sa;Password=YourStrong@Passw0rd
    networks:
      - backend_network
    depends_on:
      - db

  db:
    image: mcr.microsoft.com/mssql/server:2022-latest
    environment:
      - ACCEPT_EULA=Y
      - SA_PASSWORD=YourStrong@Passw0rd
    ports:
      - "1433:1433"
    networks:
      - backend_network

networks:
  backend_network:
```

Note on Connection Strings: Notice how the connection string uses `Server=db` . Because both containers are on `backend_network` , Docker resolves the hostname "db" to the SQL Server container's IP address.

6. Docker Network Commands

Networks allow your ASP.NET Core Web API to securely talk to Redis, SQL Server, or other microservices.

- `docker network create my_microservices_net` : Creates a new bridge network.
- `docker run --network my_microservices_net ...` : Attaches a container to the network on startup.
- `docker network ls` : Views all networks.

7. Questions and Notes

Q: Should I use HTTPS inside the Docker container?

A: Typically, no. It's best practice to run Kestrel over HTTP inside the container and use a Reverse Proxy (like Nginx, Traefik, or an Azure App Service load balancer) to handle SSL termination. The proxy then forwards HTTP traffic to your container.

Q: How do I handle Entity Framework Core Migrations in Docker?

A: Do not run `dotnet ef database update` inside the `Dockerfile` build steps, as the database container might not be running yet. Instead, either run a standalone migration container upon deployment, or apply migrations programmatically on app startup (only recommended for dev/staging).

Final .NET Developer Notes:

- Always use a `.dockerignore` file and include `bin/` and `obj/` . If you accidentally copy host-compiled binaries into a Linux container, it will cause conflicts.
- Use `UserAppHost=false` during publish in your Dockerfile to prevent generating unnecessary OS-specific executables.